

Aula 10

*DataPrev (Perfil 3: Desenvolvimento de
Software) Banco de Dados - 2026
(Pós-Edital)*

Autor:

Lucas Fargoni Di Ianni

10 de Julho de 2026

Índice

1) 13.2 Banco de Dados In-Memory - Teoria	3
2) 13.2 Banco de Dados In-Memory - Questão Comentadas - MULTIBANCAS	8
3) 13.2 Banco de Dados In-Memory - Lista de Questões - MULTIBANCAS	10

BANCO DE DADOS EM MEMÓRIA

Introdução

Pessoal, um banco de dados em memória (in-memory database, IMDB), ou banco de dados em tempo real (real time database, RTDB), é um sistema gerenciador de banco de dados que mantém os dados principalmente na memória principal do computador (a RAM), ao invés de os armazenar em disco (HDD ou SSD).

Em outras palavras, enquanto um SGBD tradicional usa o disco como local primário de armazenamento de dados, o banco em memória inverte essa lógica e trata a RAM como o repositório principal dos dados.

Certo, e por que gostaríamos de fazer isso, Professor? Porque assim temos um desempenho de velocidade de acesso aos dados muito superior. Isso porque acessar um dado na RAM é cerca de 1.000 vezes mais rápido que o acessar em um SSD e 100.000 vezes mais rápido que o acessar em um disco rígido (HDD).

Como consequência desse benefício, os bancos de dados em memória são desenvolvidos sem precisar implementar as diversas táticas de aumento de velocidade dos bancos de dados tradicionais, como lidar com índices em estruturas de árvores B/B+ e gerenciar um buffer de dados.

Assim sendo, bancos de dados em memória dispõe de mais poder de processamento da CPU para lidar com o que de fato importa, que é consultar e manipular os dados. Consequentemente, seu desempenho "natural" já é superior ao de um banco de dados padrão.

De todo modo, a ideia de manter dados na RAM existe conceitualmente desde os anos 1980, porém só ganhou tração de fato a partir de meados da década de 2000 devido a superação de algumas restrições.

Primeiramente, foi necessário evoluirmos processadores de 32 bits para 64 bits. Isso porque, a quantidade de endereços de memória de um processador de 32 bits é de aproximadamente $2^{32} \approx 4$ gigabytes de RAM, sendo inviável de trabalhar comercialmente com tão pouco espaço. Com 64 bits, chegamos a incríveis terabytes (2^{64}) de espaço disponível em RAM, viabilizando bancos em memória como produto comercial.

Além disso, ao longo dos anos 2000, o preço da RAM caiu drasticamente, tornando economicamente viável para empresas comuns montarem servidores para isso (e mais tarde, a computação em cloud intensificou ainda mais esse processo).

Principais aplicações

Por lógica, podemos concluir que bancos em memória são escolhidos quando o tempo de resposta é crítico.

Mas saibam que, com "crítico", queremos dizer que queremos tanto baixa velocidade de acesso, quanto previsibilidade nessa velocidade e também quantidade de informações trafegadas.

O problema é que bancos de dados implantados em disco rígido sofrem de seek time (tempo de busca) variável, já que, em um disco mecânico, o tempo de acesso depende da velocidade que o "braço" do HDD alcança onde o dado está fisicamente armazenado.

Dessa forma, a depender da localização deles, o tempo de acesso varia, tornando o seu desempenho irregular e difícil de prever.

Na memória, não existe essa parte móvel (braço) e nem noção de "distância física" relevante para o acesso, já que qualquer endereço é alcançado em essencialmente o mesmo tempo (inclusive, é esse o motivo do nome RAM - "Random Access Memory", memória de acesso aleatório).

Vale saber que o atraso entre o pedido de acesso a um dado e a resposta da aplicação é chamada de latência. Assim, bancos de dados em memória conseguem uma performance memorável quando analisamos o tempo de espera entre a pergunta e a resposta, minimizando o intervalo em que a aplicação aguarda a resposta.

Embora a latência seja a medida que captura o tempo de resposta, a criticidade da solução exige também análise do throughput do sistema, ou seja, quantas operações o sistema consegue processar por unidade de tempo. De todo modo, banco de dados em memória apresentam alto throughput, já que executam muitas operações por período de tempo.

Assim sendo, na prática utilizamos bancos de dados em memória quando queremos respostas e análises em "tempo real", como em sistemas de telecomunicações, jogos online, trading financeiro e cache de aplicações e sessões web.

Cache

Devido à sua importância, o uso de bancos de dados de memória para cache merece atenção especial.

O banco de cache nada mais é que uma camada sobreposta ao banco de dados tradicional, intermediando o acesso a ele.

Assim, quando uma requisição chega, primeiro consultamos o cache (essa camada rápida) e, se a resposta estiver lá, ótimo (chamamos isso de cache hit), retornamos os dados prontamente. Se não estiver (cache miss), o sistema vai buscar no armazenamento principal lento e, regra geral, guardamos uma cópia no cache para "não perder tempo" da próxima vez.

Então, percebam alguns pontos importantes dessa explicação.

Primeiro, o cache não guarda todos os dados (nem capacidade pra isso ele tem). O que guarda é provavelmente o que será requisitado de novo em breve. Ou seja, os dados com alto índice de reuso.

Segundo, o cache é transiente. Ou seja, os dados em cache são, por natureza, descartáveis. Eles podem ser apagados, expirados ou substituídos a qualquer momento sem prejuízo, porque a fonte da verdade está em outro lugar (no banco principal).

Finalmente, a utilidade do cache é proporcional ao quanto ele evita o acesso ao banco lento. Ou seja, não basta a rapidez do cache em processar requisições, é preciso que ele de fato consiga manter os dados úteis.



DevOps ("Dev" de *development* e "Ops" de *operations*) é uma **cultura que promove a união de pessoas, processos e tecnologias para fornecer continuamente valor de forma co**

Ponto fraco: Volatilidade

A volatilidade se apresenta como a moeda de troca exigida pela RAM para sua alta performance. Isso porque, se a energia acaba ou o sistema reinicia, todo o conteúdo é perdido.

Porém, devido à volatilidade, em teoria perdemos a propriedade de durabilidade (o "D" do ACID), que garante que dados confirmados (commit) sobrevivam a falhas. Ou seja, em caso de desastre, simplesmente perdemos nosso banco de dados em memória.

Assim, na prática, usamos esse tipo de bancos em memória (que deliberadamente abre mão da durabilidade) como cache puro, porque, nessa situação, perder podemos reconstruir o banco a partir dos dados da fonte original. É justamente por isso que dizemos que "caching troca durabilidade por tempo de resposta".

Mas alguns fabricantes preferem por mitigar esse problema, agindo de várias formas combinadas:

Técnica	Explicação	Desvantagem
Snapshots (checkpoints)	Cópias periódicas de todo o estado da memória são gravadas em disco. Em caso de falha, o sistema recarrega o último snapshot.	Tudo que aconteceu após o último snapshot pode ser perdido.
Transaction logging	Cada operação é registrada sequencialmente em um arquivo de log em disco antes de ser confirmada. Na recuperação, o log é reaplicado sobre o último snapshot (base do write-ahead logging).	Adiciona latência de escrita por exigir gravação em disco a cada commit, e a recuperação pode ser demorada se o log for extenso.
Replicação	Manter cópias dos dados em outras máquinas. Se um nó cai, outro assume.	Não protege contra falhas que atinjam todos os nós simultaneamente (queda de energia geral, desastre no datacenter).
Memória não-volátil (NVRAM/NVDIMM) ou RAM com bateria	Hardware especial que preserva o conteúdo da memória mesmo sem energia, oferecendo durabilidade sem precisar ir ao disco.	Custo elevado, disponibilidade limitada e capacidade menor que a da RAM tradicional ou do disco.

Armazenamento e índices

Em um banco de dados relacional tradicional, trabalhamos com o índice como uma estrutura de árvore B + (B-tree), preocupada em minimizar o número de blocos de disco que precisam ser lidos para localizar um registro.

Em resumo, nessa estrutura cada nó da árvore é desenhado para caber exatamente em um bloco de disco, e a árvore é mantida rasa de propósito (geralmente 3 ou 4 níveis) para que poucas leituras de disco bastem.

Assim, para encontrar um registro, são necessárias várias etapas, somando-se em uma operação custosa.

Já em um banco em memória, esse problema não existe, porque usamos estruturas radicalmente diferentes, como:

Técnica	Explicação
Tabelas hash	Dão acesso em tempo essencialmente constante, uma operação de hash e um salto direto, sem percorrer nenhuma árvore. É uma estrutura típica de caches chave-valor como Redis e Memcached.
Árvores T	Projetadas especificamente para residência em memória (combinam características de árvores AVL e árvores B), otimizadas para baixo custo de comparação na CPU em vez de baixo número de leituras em disco.
Skip lists	Busca rápida de implementação mais simples.

QUESTÕES COMENTADAS - COMENTADAS



1. (CEBRASPE (CESPE)/TCE AC/Gestão de Dados/2024) Esses bancos de dados são caracterizados por armazenar e manipular dados diretamente na memória principal do sistema, proporcionando acesso mais rápido aos dados, em comparação aos bancos de dados tradicionais que utilizam armazenamento em disco.

Comentários:

A afirmação descreve exatamente o que é um banco de dados em memória. A ideia central é justamente essa: em vez de guardar os dados no disco (HDD ou SSD) como fazem os bancos tradicionais, esse tipo de banco mantém tudo na memória RAM. E por que isso importa? Porque acessar dado na RAM é cerca de mil vezes mais rápido que no SSD e cem mil vezes mais rápido que no HDD. Por isso, o ganho de velocidade citado na questão está correto.

Gabarito: Certo

2. (CEBRASPE (CESPE)/TSE/Programação de Sistemas/2024) Os bancos de dados em memória apresentam baixa latência, respostas em tempo real e baixo throughput.

Comentários:

O erro está na parte final. Bancos de dados em memória realmente entregam baixa latência e respostas em tempo real, mas o throughput deles é alto, não baixo. Throughput é a quantidade de operações processadas por unidade de tempo, e justamente por serem rápidos, esses bancos conseguem executar muitas operações em pouco tempo.

Gabarito: Errado

3. (CEBRASPE (CESPE)/TSE/Apoio Especializado/2024) Bancos de dados em memória são inerentemente mais seguros contra falhas de energia que bancos de dados em disco, pois eles armazenam todos os dados em uma cache volátil que é sincronizada automaticamente com o armazenamento persistente.

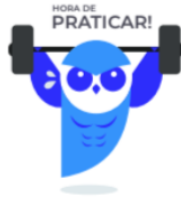
Comentários:

A afirmação inverte completamente a realidade. Bancos de dados em memória são justamente menos seguros contra falhas de energia, e não mais seguros. Isso acontece porque a RAM é volátil: se a energia acaba ou o sistema reinicia, todo o conteúdo é perdido. Esse é o ponto fraco desse tipo de banco, não uma vantagem. Além disso, não existe essa tal "sincronização automática" com o armazenamento persistente como regra. O que existe são técnicas opcionais

que alguns fabricantes adotam para mitigar a volatilidade, como snapshots periódicos, transaction logging ou replicação, mas nenhuma delas é uma sincronização automática nativa e garantida.

Gabarito: Errado

LISTA DE QUESTÕES



1. (CEBRASPE (CESPE)/TCE AC/Gestão de Dados/2024) Esses bancos de dados são caracterizados por armazenar e manipular dados diretamente na memória principal do sistema, proporcionando acesso mais rápido aos dados, em comparação aos bancos de dados tradicionais que utilizam armazenamento em disco.
2. (CEBRASPE (CESPE)/TSE/Programação de Sistemas/2024) Os bancos de dados em memória apresentam baixa latência, respostas em tempo real e baixo throughput.
3. (CEBRASPE (CESPE)/TSE/Apoio Especializado/2024) Bancos de dados em memória são inerentemente mais seguros contra falhas de energia que bancos de dados em disco, pois eles armazenam todos os dados em uma cache volátil que é sincronizada automaticamente com o armazenamento persistente.

GABARITO

1. Certo

2. Errado

3. Errado

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.